

Environment variables

The backend reads settings through `backend/app/core/config.py` (Pydantic `BaseSettings`). Values come from the process environment, then from `backend/.env`. The env file path is **relative to the working directory**, so start `uvicorn`, `celery`, `alembic` and `pytest` from `backend/`. Names are case-insensitive. Copy `backend/.env.example` to `backend/.env`; never commit `.env`.

Required below means "the pipeline does not work without it", not "the process refuses to start" — most services are built lazily, so a missing value surfaces when first used.

Backend — required for a working local pipeline

Variable	Default	Purpose	Where to get it	If missing
<code>DATABASE_URL</code>	<code>postgresql+psycopg2://docauth:docauth@localhost:5432/docauth</code>	The owner connection — used only by Alembic and <code>seed.py</code> (needs <code>CREATEROLE</code> , not superuser). Its host/port/database are also used for the two app roles below	Matches the Postgres in <code>docker-compose.yml</code> (default works as-is)	Migrations fail; API/worker can't build their URLs
<code>DATABASE_APP_USER</code> / <code>DATABASE_APP_PASSWORD</code>	<code>fddt_app</code> / <code>fddt_app_local</code>	The role every company request and pipeline task connects as (NOBYPASSRLS — Row-Level Security applies). The migration creates it with this password	Choose a long random password before running <code>alembic upgrade head</code>	Every request fails to connect
<code>DATABASE_PLATFORM_USER</code> / <code>DATABASE_PLATFORM_PASSWORD</code>	<code>fddt_platform</code> / <code>fddt_platform_local</code>	The role for sign-in, platform-admin screens and usage reconciliation (all rows through the <code>platform_all</code> policy; no BYPASSRLS)	Same	Login and platform screens fail
<code>CELERY_BROKER_URL</code>	<code>redis://localhost:6379/0</code>	Celery broker	The Redis in <code>docker-compose.yml</code>	Uploads succeed but nothing is processed; documents stay pending
<code>CELERY_RESULT_BACKEND</code>	<code>redis://localhost:6379/1</code>	Celery results	Same Redis, DB 1	Worker errors on result storage
<code>JWT_SECRET_KEY</code>	<code>changeme-in-.env</code>	Signs login tokens (HS256)	Generate a long random string, e.g. <code>python -c "import secrets; print(secrets.token_urlsafe(48))"</code>	Works with an insecure default — always change it; changing it logs everyone out
<code>AZURE_STORAGE_CONNECTION_STRING</code>	<i>(none)</i>	Blob Storage account for originals, signature crops and reports	Azure portal → Storage account → Access keys → connection string	Upload and file-url endpoints return 503 with an explanatory message

Variable	Default	Purpose	Where to get it	If missing
<code>AZURE_STORAGE_CONTAINER_NAME</code>	<code>documents</code>	Blob container (must be private)	Create it in the storage account	Uploads fail
<code>AZURE_DOCUMENT_INTELLIGENCE_ENDPOINT</code>	<i>(none)</i>	OCR/layout (<code>prebuilt-layout</code>)	Azure portal → Document Intelligence resource → <i>Keys and Endpoint</i>	Every document ends <code>failed</code> at extraction
<code>AZURE_DOCUMENT_INTELLIGENCE_KEY</code>	<i>(none)</i>	Same	Same	Same
<code>AZURE_OPENAI_KEY</code>	<i>(none)</i>	Classification, extraction, issuer semantic match, visual review, signature detect/compare	Azure portal (or AI Foundry) → your OpenAI resource → <i>Keys</i>	Extraction fails (document <code>failed</code>); visual review/signature checks fail
<code>AZURE_OPENAI_ENDPOINT</code>	<i>(none)</i>	Resource endpoint. Accepts either <code>https://<res>.openai.azure.com/</code> or an AI Foundry project URL <code>https://<res>.services.ai.azure.com/api/projects/<name></code> (normalized to the origin)	Same	Same
<code>AZURE_OPENAI_DEPLOYMENT_NAME</code>	<i>(none)</i>	The model deployment to call. Must be vision-capable : its name must contain <code>gpt-4o</code> , <code>gpt-4.1</code> , <code>gpt-4-turbo</code> or <code>gpt-4-vision</code> , or the visual review and signature features raise <code>LLMConfigurationError</code>	Name you gave the deployment in Azure	Same
<code>AZURE_OPENAI_MAX_REQUESTS_PER_MINUTE</code>	<code>60</code>	Global cap on Azure OpenAI requests across every worker (Redis limiter). Set to ~80 % of the deployment's RPM	Azure portal → deployment → <i>Rate limit</i> , or the <code>x-ratelimit-limit-requests</code> response header	Default 60 may under- or overshoot your quota (429s or idle capacity)
<code>AZURE_OPENAI_MAX_TOKENS_PER_MINUTE</code>	<code>0</code> (off)	Global token cap (prompt + <code>max_tokens</code> reservation per call). Usually the binding limit. Set to ~80 % of the deployment's TPM	<code>x-ratelimit-limit-tokens</code> header / portal	Without it, bursts exceed TPM → 429s

There is **no local/offline substitute** for storage, OCR or the LLM (no Azurite/mock backend is wired in); a real Azure account is needed to exercise the pipeline. The unit tests mock all three, so they do not call Azure.

Backend — optional / tuning

Variable	Default	Purpose
<code>ENVIRONMENT</code>	<code>local</code>	Label shown in <code>/health</code> and the OpenAPI description
<code>JWT_ALGORITHM</code>	<code>HS256</code>	JWT algorithm

Variable	Default	Purpose
<code>JWT_ACCESS_TOKEN_EXPIRE_MINUTES</code>	480 (8 h)	Login token lifetime
<code>AZURE_OPENAI_API_VERSION</code>	2024-10-21	Azure OpenAI REST API version
<code>AZURE_OPENAI_REQUEST_TIMEOUT_SECONDS</code>	60.0	Per-call LLM timeout. Kept short on purpose: a hung call holds a worker thread (and its rate-limiter reservation)
<code>AZURE_DOCUMENT_INTELLIGENCE_MAX_CALLS_PER_SECOND</code>	10.0	Global cap on Document Intelligence calls (Azure's default SO limit is 15 TPS)
<code>AZURE_DOCUMENT_INTELLIGENCE_STYLE_FONT</code>	true	Request Layout's font-style add-on (<code>styleFont</code>), which the font consistency check uses on scanned pages. Billed by Azure as an add-on per page; <code>false</code> leaves fonts on scans to the visual review only
<code>DATABASE_POOLER_HOST / DATABASE_POOLER_PORT</code>	(unset) / 6432	When set, the two app roles connect through PgBouncer (transaction pooling) and each process keeps no pool of its own. Recommended in production (processing-queues.md)
<code>DB_POOL_SIZE / DB_MAX_OVERFLOW</code>	5 / 10	SQLAlchemy pool per role per process when connecting directly (no PgBouncer)
<code>DEFAULT_MAX_FILE_SIZE_MB / DEFAULT_MAX_ZIP_SIZE_MB</code>	10 / 300	Upload limits a new company is created with. Limits are per company (<code>companies.max_file_size_mb / max_zip_size_mb</code> , changed by a platform admin under Platform > Companies); changing these defaults never changes an existing company. The frontend reads the company's real limits from <code>GET /auth/me/upload-limits</code>
<code>BULK_UPLOAD_MAX_ENTRIES</code>	25000	Safety cap on entries (files + folders) in one zip. A 300 MB zip of small born-digital PDFs holds about 11,000 entries
<code>BULK_UPLOAD_CASE_WARNING_THRESHOLD</code>	100	Above this many cases the UI warns; the zip is still accepted
<code>DEFAULT_COMPANY_NAME</code>	Default Company	Name of the company the multi-tenancy migration assigns pre-existing data to (used once, by the migration)
<code>PLATFORM_ACCESS_AUDIT_DEDUP_SECONDS</code>	0	0 = every platform-admin read of company data writes its own (platform-only) audit row; > 0 collapses identical reads within that window
<code>EXTRACTION_WORKER_CONCURRENCY / VISION_WORKER_CONCURRENCY</code>	4 / 4	Threads per extraction / vision worker (<code>start-worker.sh</code>). Size to just saturate the Azure caps

Variable	Default	Purpose
<code>FORENSICS_WORKER_CONCURRENCY</code>	<code>0</code>	Prefork processes per forensics worker; 0 = one per CPU of the container's limit
<code>FAIR_SHARE_BUCKET_SIZE</code>	<code>5</code>	A company's tasks drop one priority level per this many outstanding tasks on a queue
<code>QUEUE_ALERT_OLDEST_WAITING_SECONDS</code>	<code>300</code>	The per-minute <code>queue_metrics</code> job logs a <code>WARNING queue_alert {json}</code> line when a queue's oldest waiting task is older than this; 0 = off
<code>USAGE_RECONCILIATION_HOUR_UTC</code>	<code>2</code>	Hour (UTC) of the nightly usage-counter reconciliation (Celery beat)
<code>STUCK_DOCUMENT_PENDING_MINUTES / STUCK_DOCUMENT_PROCESSING_MINUTES</code>	<code>10 / 30</code>	Stuck-document recovery (every 5 minutes, <code>app/tasks/stuck_documents_task.py</code>): a document still <code>pending</code> after this long, or <code>processing</code> with no update for this long, while the extraction queue is empty and idle , lost its tasks (Redis restart/outage) and is queued again. <code>0</code> disables that half
<code>STUCK_DOCUMENT_MAX_REQUEUES</code>	<code>3</code>	Re-queues per document (counted from its <code>document_requeued</code> audit rows) before it is marked <code>failed</code> with an explanation
<code>STUCK_DOCUMENT_BATCH_SIZE</code>	<code>200</code>	Most documents re-queued per run
<code>LOGIN_MAX_FAILED_ATTEMPTS / LOGIN_LOCKOUT_MINUTES</code>	<code>5 / 15</code>	Sign-in throttling (<code>app/services/login_throttle.py</code> , Redis): after this many failed sign-ins for one email address, it is refused (HTTP 429 + <code>Retry-After</code>) for this many minutes. Counted per address (unknown addresses too), cleared by a successful sign-in. <code>0</code> disables
<code>LOGIN_MAX_ATTEMPTS_PER_IP_PER_MINUTE</code>	<code>20</code>	Sign-in attempts of any outcome per client IP per minute (best effort: behind the proxy chain a client can influence <code>X-Forwarded-For</code>). <code>0</code> disables. Both limits fail open if Redis is unreachable
<code>SEED_ADMIN_EMAIL / SEED_ADMIN_PASSWORD</code>	<code>admin@example.com / ChangeMe123!</code>	Read by <code>seed.py</code> (not by the app): the platform admin it creates — and resets on every run (<code>start-api.sh</code> runs it). Set both in production
<code>CELERY_POOL</code>	<code>per queue</code>	Read by <code>start-worker.sh</code> : overrides the pool type (<code>threads</code> for extraction/vision, <code>prefork</code> for forensics)
<code>CELERY_BEAT_SCHEDULE_FILE</code>	<code>/tmp/celerybeat-schedule</code>	Read by <code>start-beat.sh</code> (beat + the one-slot worker for <code>housekeeping_queue</code>)
<code>ISSUER_FUZZY_MATCH_THRESHOLD</code>	<code>85.0</code>	rapidfuzz WRatio score (0–100) an extracted issuer must reach to match the registry (05)

Variable	Default	Purpose
<code>METADATA_FORENSICS_MOD_DATE_THRESHOLD_SECONDS</code>	1.0	Jitter tolerance before "modified after created" flags (06)
<code>METADATA_FORENSICS_EDITING_SOFTWARE_NAMES</code>	photoshop,gimp,illustrator,affinity,coreldraw,paint.net,pixlr,canva,inkscape	Comma-separated, case-insensitive substrings that mark a Producer/Creator as an editing tool
<code>METADATA_FORENSICS_PDF_EDITOR_NAMES</code>	ilovepdf,smallpdf,sejda,pdfescape,pdf-xchange editor,phantompdf,foxit pdf editor,nitro pro,nitro pdf pro,pdfelement,soda pdf,sodapdf,pdffiller,dochub,pdfcandy,pdfcandy	Same, for PDF editors (online and desktop); weighted lower than image editors
<code>DUPLICATE_HASH_HAMMING_THRESHOLD</code>	5	Max pHash Hamming distance (0–64) for a near-duplicate (10)

Risk **weights, severities and tier thresholds are not environment variables** — they live in the database, per company, and are edited in Settings > Risk Rules (and the platform template under Platform > Rule templates; see 11).

Backend — declared but UNUSED (do not need a value)

Variable	Would be for	Status
<code>AZURE_AI_VISION_ENDPOINT</code> , <code>AZURE_AI_VISION_KEY</code>	Signature/stamp detection, logo matching	Never read — detection uses the Azure OpenAI vision model
<code>AI_CONTENT_DETECTION_ENDPOINT</code> , <code>AI_CONTENT_DETECTION_KEY</code>	Content Safety / Hive	Never read — see 08
<code>EMAIL_SERVICE_CONNECTION_STRING</code> , <code>EMAIL_FROM_ADDRESS</code>	Transactional email	Never read — notifications are not built
<code>API_V1_PREFIX</code> (/api/v1)	URL prefix	Never read — routes have no prefix
<code>REDIS_URL</code>	Redis	Never read — Celery, the rate limiter, the fair-share counters and the queue monitor all use <code>CELERY_BROKER_URL</code>

docker-compose (optional overrides)

Read by `docker-compose.yml` from the shell or a root `.env` ; defaults shown.

Variable	Default	Purpose
<code>POSTGRES_USER</code> / <code>POSTGRES_PASSWORD</code> / <code>POSTGRES_DB</code>	<code>docauth</code> / <code>docauth</code> / <code>docauth</code>	Postgres container credentials. If you change them, update <code>DATA_BASE_URL</code> to match
<code>POSTGRES_PORT</code>	5432	Host port
<code>REDIS_PORT</code>	6379	Host port

Variable	Default	Purpose
<code>PGBOUNCER_PORT</code>	<code>6432</code>	Host port of PgBouncer (<code>pgbouncer/pgbouncer.ini</code> , <code>pgbouncer/userlist.txt</code> — dev passwords only)
<code>FORENSICS_WORKER_CPUS</code>	<code>2</code>	CPUs per <code>forensics-worker</code> replica (<code>--profile workers</code>)

Frontend

Variable	Default	Purpose
<code>VITE_API_BASE_URL</code>	<code>/api</code>	Base URL the SPA calls (build time). The default relies on a proxy that maps <code>/api/*</code> → the API with the prefix stripped — the Vite dev server locally, the ingress in production (<code>production-deployment.md</code>). Setting it to a different origin instead requires CORS on the backend, which currently has no CORS middleware

No other frontend variables exist. Login state is a JWT in `localStorage` .

Secrets checklist

`AZURE_STORAGE_CONNECTION_STRING` , `AZURE_DOCUMENT_INTELLIGENCE_KEY` , `AZURE_OPENAI_KEY` , `JWT_SECRET_KEY` , the owner database password (in `DATABASE_URL`), `DATABASE_APP_PASSWORD` , `DATABASE_PLATFORM_PASSWORD` and `SEED_ADMIN_PASSWORD` are the secrets (plus PgBouncer's `userlist.txt`). Keep them out of the repo (`.env` is git-ignored) and, on Azure, in Key Vault referenced from the container apps (see `production-deployment.md`).